

# CRUD i FK, PRG

## Wykłady 9

Bartosz Lauks

7 maja 2026



① CRUD i FK

② (PRG) Post/Redirect/Get

## ① CRUD i FK

## ② (PRG) Post/Redirect/Get

# CRUD i FK

## Tworzenie CRUD posty i komentarze

- Na potrzeby zajęć i omówienia kolejnych etapów wykładu zaimplementujemy nową funkcjonalność **CRUD**.
- to akronim oznaczający podstawowe operacje wykonywane na danych w aplikacjach, szczególnie w kontekście baz danych. Rozwinięcie tego skrótu to:
  - **C – Create** (tworzenie) – dodawanie nowych danych.
  - **R – Read** (odczyt) – pobieranie i wyświetlanie danych.
  - **U – Update** (aktualizacja) – modyfikowanie istniejących danych.
  - **D – Delete** (usuwanie) – kasowanie danych.
- Przykładem naszego CRUD-a będzie system postów z dołączonymi do nich komentarzami, które użytkownik będzie mógł dodawać.

# CRUD i FK

## Nowe tablice w bazie danych

- Pierwszym elementem będzie dodanie do istniejącej bazy nowych tabel:
- **post** - zawierać będzie dane o postach:
  - **title** (**varchar(255)**) - tytuł postu.
  - **description** (**text**) - opis postu.
  - **created\_at** (**timestamp**) - znacznik czasu określający dodane postu.
- **comments** - tabele przechowywać będzie dane komentarzy pisanych pod postem. Zawierać będzie pola:
  - **content** (**text**) - zawartość komentarza.
  - **created\_at** (**timestamp**)- znacznik czasu określający dodane komentarza.
  - **post\_id** (**int(11)**) - przechowywać będzie numer **id** postu do którego dołączony będzie komentarz.
  - **author\_id** (**int(11)**) - podobna sytuacja jak w przypadku pola **post\_id**, ale w tym przypadku relacje będzie z tablicą **user**.

## CRUD i FK

## Przypomnienie o relacjach w bazach danych

- W bazie danych odzwierciedlamy relację za pomocą klucza obcego (**foreign key, FK**).
- **Klucz obcy** to pole, które wskazuje na **klucz główny** (zwykle id) w innej tabeli.
- W naszym przypadku tabela **comments** może mieć pole o nazwie np. **user\_id**.
- To pole przechowuje **id użytkownika**, który jest autorem komentarza.
- W ten sposób tworzymy relację typu **wiele-do-jednego (many-to-one)** między komentarzami a użytkownikami.
- Dzięki temu zachowujemy spójność danych i możemy w łatwy sposób pobierać informacje o autorze komentarza lub wszystkie komentarze danego użytkownika.

## CRUD i FK

## Tworzenie relacji między tabelami w PHPMyAdmin

- Aby utworzyć relację między tabelami w **PHPMyAdmin** należy przejść do struktury tablicy, która będzie przechowywała w polach id obcej tablicy. (comments)
- Następnie w "**Structure**" przechodzimy do "**Relation view**"
- W "**Constraint properties**" podajemy nazwę dla klucza obcego.
- Kolejno musimy wybrać opcję dla pól "ON DELETE"i "ON UPDATE"dotycząc zachowania relacji (kluczy obcych) między tabelami w bazie danych MySQL.
- Określają one, co ma się stać z rekordami zależnymi, gdy zostanie usunięty lub zaktualizowany rekord w tabeli nadrzędnej (czyli tej, do której odnosi się klucz obcy).

## CRUD i FK

## ON DELETE - możliwe opcje

- Dla pola **ON DELETE** mamy kilka opcji:
  - **RESTRICT** - Zabrania usunięcia lub aktualizacji, jeśli istnieją powiązane rekordy.
  - **CASCADE** - Automatycznie usuwa/aktualizuje powiązane rekordy.
  - **SET NULL** - Ustawia wartość pola klucza obcego na NULL, jeśli to możliwe.
  - **NO ACTION** - Działa jak RESTRICT, ale formalnie pozwala na opóźnione sprawdzenie.
  - **SET DEFAULT** - Ustawia wartość domyślną (rzadko używane – MySQL ma ograniczenia).



## CRUD i FK

## ON UPDATE – możliwe opcje

- Dla pola **ON UPDATE** mamy kilka opcji:
  - **CASCADE** - Gdy klucz główny w tabeli nadrzędnej zostanie zmieniony, automatycznie zmienia się też w tabeli zależnej.
  - **RESTRICT** - Zmiana zostaje zablokowana, jeśli istnieją powiązane rekordy.
  - **SET NULL** - W tabeli zależnej pole klucza obcego zostaje ustawione na NULL.
  - **NO ACTION** - Jak RESTRICT, ale działanie może być opóźnione.
  - **SET DEFAULT** - Ustawia wartość domyślną (mało używane i nie zawsze wspierane w MySQL).

## CRUD i FK

## Tworzenie relacji między tabelami w PHPMyAdmin cd.

- Kolejnym polem do wyboru będzie kolumna, która przechowywać będzie w relacji. (**post\_id**)
- Następnie należy wybrać bazę danych, odpowiednią tabelę oraz pole, które będzie powiązane relacją z bieżącą tabelą. (**post, id**)
- Po utworzeniu **foreign key** można sprawdzić w tablicy **Indexes**.
- Powinien pojawić się tak kolejny **index FK**

## CRUD i FK

## Strona z listą postów

- Po wprowadzeniu zmian w schemacie bazy danych oraz dodaniu przykładowego posta i kilku komentarzy (za pomocą phpMyAdmin), co umożliwiło przetestowanie poprawności relacji, możemy przejść do projektowania layoutu odpowiedzialnego za wyświetlanie postów.
- Jedyne co musimy zrobić to dodać połączenie bazy danych za pomocą: **require 'dbPDO.php'** i stworzyć zapytanie **SELECT**, które zwróci nam wszystkie posty w bazie danych.
- Następnie za pomocą pętli **while** iterując po tablicy postów wyświetlimy o nich postach na stornie.

Przykład: <localhost/lecture9/posts.php>

# CRUD i FK

## Funkcja nl2br()

- Warto wspomnieć o funkcji `nl2br()`.
- Odpowiada za konwertowanie znaków nowej linii (np. `\n`) w tekście na znaczniki HTML `<br>`.

## CRUD i FK

## Strona z komentarzami

- Aby przejść do wyświetlania komentarzy musimy w jakiś sposób przekazać skryptowi id postu, który nas interesuje.
- Ponieważ będziemy starać się, aby nasz program dostosował swoje działanie w zależności od zmieniających się danych wejściowych.
- Użyjemy **query-param**, aby przesłać id postu, którego komentarz nas interesują.
- W ten sposób ten sam skrypt zwracał będzie komentarze innych postu w zależności od podanego parametru.

## CRUD i FK

## Strona z komentarzami cd.

- W pliku `posts.php` na podstawie `id` pobranego z bazy danych dodany do hiperłącza unikalny parametr postu.  
`<a href="post.php?id=<?php echo $post['id'] ?>»Czytaj więcej</a>`
- Dlatego w `post.php` należy przechwycić go przy wykorzystaniu **super globalnej** `$_GET`.
- W taki sposób skrypt na podstawie parametru pobierze komentarz z tabeli `comments` wykorzystując powiązanie **klucza obcego**.

## CRUD i FK

## Strona z komentarzami cd.

- Teraz zostaje tylko wyświetlenie jeszcze raz danych postu wybranego przez użytkownika oraz przy pomocy pętli (**foreach**) wyświetlenie wszystkich komentarzy.
- Dzięki dodania do **SQL query** klauzuli **ORDER BY c.created\_at DESC** komentarz zostały zwrócone przez baza danych malejąco.
- Co na podstawie daty oznaczać będzie, że najnowsze komentarzy pojawiać się będą na początku sekcji.
- Na samym dole strony zostanie dodaony formularz, aby umożliwić dodawanie komentarzy do danego postu.

Przykład: <localhost/lecture9/post.php>

## CRUD i FK

## Formularz dodawania komentarza

- Po wypełnieniu formularza przez użytkownika zostanie on przesłany do pliku [commentCreate.php](#) w którym znajduje się skrypt odpowiedzialny za dodanie komentarza.
- Tu powtarza się problem jaki sposób poinformować kolejny skrypt o jaki zasób dokładnie nam chodzi.
- Do którego posta ma zostać dodany komentarz. W tym celu wcześniej wykorzystano metodę **GET**, ponieważ zgodnie ze standardem HTTP służy ona do pobierania danych z serwera.
- Parametr identyfikujący post przekazano więc jako **query parameter**, gdyż żądanie GET nie zawiera ciała (**body**).



## CRUD i FK

## Formularz dodawania komentarza

- Teraz wysyłamy do serwera żądanie utworzenia nowego zasobu (komentarza), dlatego używamy metody **POST**.
- Treść komentarza (**content**) oraz **id posta** zostaną przekazane w ciele żądania.
- Dodatkowo pole z **id** zostanie ukryte w formularzu przy użyciu atrybutu **type=hidden**.

## CRUD i FK

## Dodawanie komentarza

- W `commentCreate.php` wystarczy tylko odczytać dane z `$_POST` i za pomocą klauzuli **SQL INSERT INTO** dodać dane do bazy.
- Ważne jest również to, że każdy komentarz pozostaje w relacji **ManyToOne** nie tylko z postem, ale także z tabelą użytkowników. Dzięki temu możemy określić, kto dodał dany komentarz.
- Istnieje kilka sposobów uzyskania **id użytkownika**:
  - 1 Wykonanie dodatkowego zapytania do bazy danych - rozwiązanie mało optymalne.
  - 2 Przesłanie identyfikatora w formularzu - rozwiązanie niebezpieczne, mogące stanowić wektor ataku (np. SQL Injection).

# CRUD i FK

## Dodawanie komentarza

- Najlepszym rozwiązaniem jest zapisanie identyfikatora użytkownika w zmiennej sesyjnej, np. `$_SESSION['user_id']`, po udanym logowaniu (plik [loginPDO.php](#), linia 46).
- Na koniec, gdy komentarz został dodany strona zwróci odpowiedzi komunikat.

Przykład: [localhost/lecture9/commentCreate.php](#)

## 1 CRUD i FK

## ② (PRG) Post/Redirect/Get

# (PRG) Post/Redirect/Get

## Problem z PGR

- Niestety obecnie nasza funkcjonalności dodania komentarzy do postu łamie zasadę **PGR** czyli **Post/Redirect/Get**.
- Musimy znów wrócić do podstawowego założenia protokołu **HTTP**.
- Klient wysyła żądanie o zasób (**request**) pod pewnym adresem URL i oczekuje rezultatu czyli odpowiedzi (**respons**).
- W aplikacjach webowych bardzo często użytkownik przesyła dane formularzem (np. logowanie, rejestracja, dodanie komentarza). Formularze te zwykle używają metody **POST**, ponieważ wysyłają dane do serwera.

(PRG) Post/Redirect/Get

## Co to PGR ?

- W takim przypadku napotykamy problem.
- Po wysłaniu formularza i odświeżeniu strony przeglądarka ponownie wysyła dane POST, co może prowadzić do:
  - Powtórzenia akcji (np. drugi raz wysłany formularz i zdublowane zamówienie, dwa komentarze itp.).
  - Komunikatu przeglądarki: Czy na pewno chcesz ponownie przesłać dane?.
- Aby sobie z tym poradzić stosuję się właśnie model (**PRG**)  
**Post/Redirect/Get**

Zadanie: Dodaj komentarz i odśwież stronę kilka razy. Ile jest komentarzy ?

# (PRG) Post/Redirect/Get

## Zasada działania PRG

- 1 Użytkownik wypełnia formularz i wysyła go metodą **POST**.
- 2 Serwer przetwarza dane (np. zapisuje do bazy).
- 3 Zamiast zwrócić stronę HTML bezpośrednio, serwer wysyła nagłówek **Location:** z kodem **302** (lub **303**), co przekierowuje użytkownika do innego adresu URL (zwykle metodą **GET**).
- 4 Przeglądarka wykonuje nowe żądanie **GET** – wyświetla wynik (np. podziękowanie, potwierdzenie).

Dzięki temu odświeżenie strony nie powoduje ponownego przesyłania danych **POST**.

# (PRG) Post/Redirect/Get

## Różnice w przekierowaniach

Warto w tym miejscu omówić różnice między kodami statusu przekierowań oraz sposób, w jaki są one interpretowane przez przeglądarki zgodnie ze standardem HTTP.

- ❶ **301 Moved Permanently** - Trwałe przekierowanie na nowy adres URL. Jest zapisywane w pamięci podręcznej. W przypadku SEO: przekazuje „moc” linków (ranking).
- ❷ **302 Found** - Tymczasowe przekierowanie. Przeglądarka nie powinna zapamiętywać nowego URL.
- ❸ **303 See Other** - Przekierowanie do innego zasobu, ale zawsze używa metody **GET**, niezależnie od oryginalnej. Stosuje wrzozec **PRG**



## (PRG) Post/Redirect/Get

### Zanotowanie PRG do systemu komentarzy

- Aby zastosować model PRG w naszej aplikacji wystarczy, dodanie tylko przekierowania z [commentCreate.php](#) z powrotem do [post.php](#)
- Ale dodatkowo zmodyfikujemy kod tak, aby dodawania komentarza wykonywało się w tym samym skrypcie.
- Wystarczy usunąć atrybut **action** z formularza. Spowoduje to że formularz zostanie wysłany pod URL w którym się sam znajduje.
- Po tej zmianie należy przenieść logikę dodawania komentarza do [post.php](#) i zabezpieczyć ją w taki sposób, aby była uruchamiana tylko po przesłaniu formularza metodą **POST**.

Dziękuję za uwagę !

Bartosz Lauks